SEGURIDAD INFORMÁTICA · UNIVERSIDAD NACIONAL

Laboratorio 6
Monitoreo de UNA Chat

Vercel, Cronitor, Sentry, OpenTelemetry, Grafana Cloud, Docker y Artillery

Estudiante: Gerald

Fecha: 30 de octubre de 2026

Repositorio:
github.com/Gerald120904/infosec-unachat-2026

Estado: avance verificado; pendientes explícitos en Sentry y Grafana Cloud.

01 · OBJETIVO

Objetivo y alcance

Instrumentar y verificar UNA Chat en su despliegue público y en un entorno local contenerizado. La práctica cubre disponibilidad, captura de errores, telemetría distribuida y carga funcional a través del protocolo real del chat.

Resultados objetivo

- Publicar el fork en Vercel con Node.js 24 y verificar el intercambio de mensajes Socket.IO sobre WebSocket.
- Monitorear el endpoint público con Cronitor y habilitar alertas.
- Preparar Sentry para registrar errores del entorno Vercel.
- Enviar logs, métricas y trazas únicamente desde la ejecución local/Docker hacia Grafana Cloud.
- Comparar 300 mensajes Socket.IO y 300 solicitudes HTTP de prueba exclusivamente contra localhost.

> **Regla de alcance:** Artillery nunca se ejecutó contra Vercel. No se enviaron logs locales, métricas ni trazas desde el runtime público Vercel hacia Grafana Cloud.

Estado comprobable

Componente	Estado
Vercel + Socket.IO	Operativo; dos clientes intercambiaron mensajes.
Cronitor	Monitor Vercel Healthy; alerta activa.
Sentry	Proyecto creado; evento real aún pendiente.
Grafana Cloud	Política OTLP limitada creada; ingestión pendiente del token local y consulta.
Docker + Artillery	Operativo; carga local completada.

02 · ARQUITECTURA

Aplicación e instrumentación

UNA Chat utiliza Express para servir la interfaz y Socket.IO para recibir el evento `Evento-Mensaje-Server` y difundir el mensaje a los clientes conectados.

Cambios implementados

- Socket.IO usa la ruta `/api/socket-io/socket.io` en cliente, servidor y pruebas, con transporte WebSocket y reconexión.
- OpenTelemetry se inicializa antes de Express/HTTP; se agregaron spans de mensajes y métricas HTTP/chat.
- Pino produce eventos estructurados con `trace_id` y `span_id`, sin registrar el texto del chat.
- Sentry Node SDK solo inicia en Vercel y requiere `SENTRY_DSN`; se desactiva el envío de PII y se filtran cookies, cabeceras y query strings.
- La ruta HTTP de laboratorio `/__lab/message` solo está disponible fuera de Vercel y reutiliza el validador real.
- Docker publica el servicio solamente en `127.0.0.1:3000`.

> **Privacidad de telemetría:** la configuración previene exportar desde Vercel. La entrega OTLP real desde Docker todavía requiere validar credenciales localmente en el archivo ignorado `.env`.

Protocolo Socket.IO verificado

El cliente envía una cadena JSON con `nombre`, `mensaje` y `color` a `Evento-Mensaje-Server`. El servidor valida el contenido y lo difunde a los clientes mediante el evento de respuesta del chat. Las pruebas validan recepción en ambos clientes.

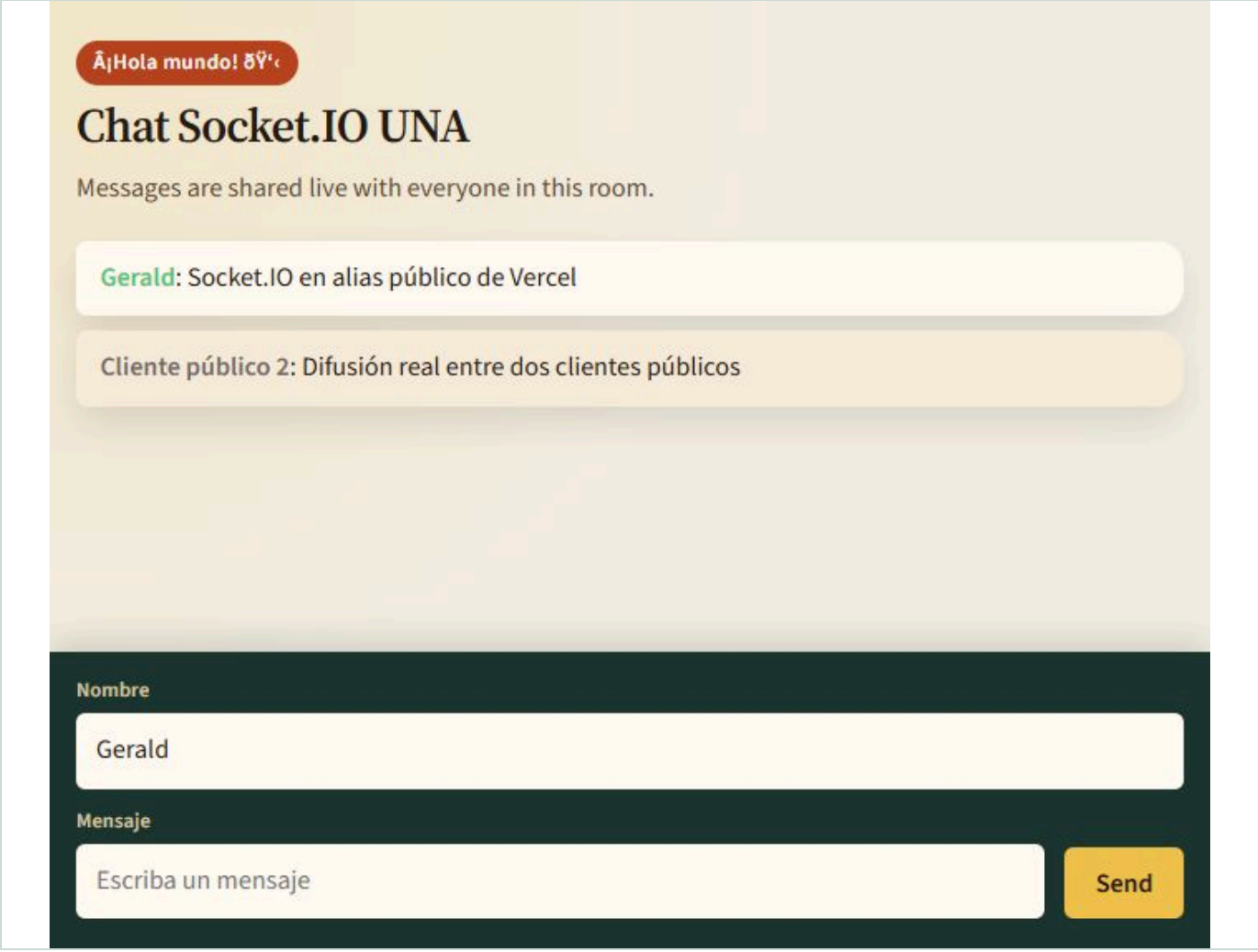
03 · VERCEL

Despliegue público

El proyecto Vercel está conectado al fork correcto y a la rama `master`. El panel mostró el deployment en estado Ready y la aplicación pública respondió correctamente.

URL pública: https://infosec-unachat-2026.vercel.app

Repositorio: Gerald120904/infosec-unachat-2026

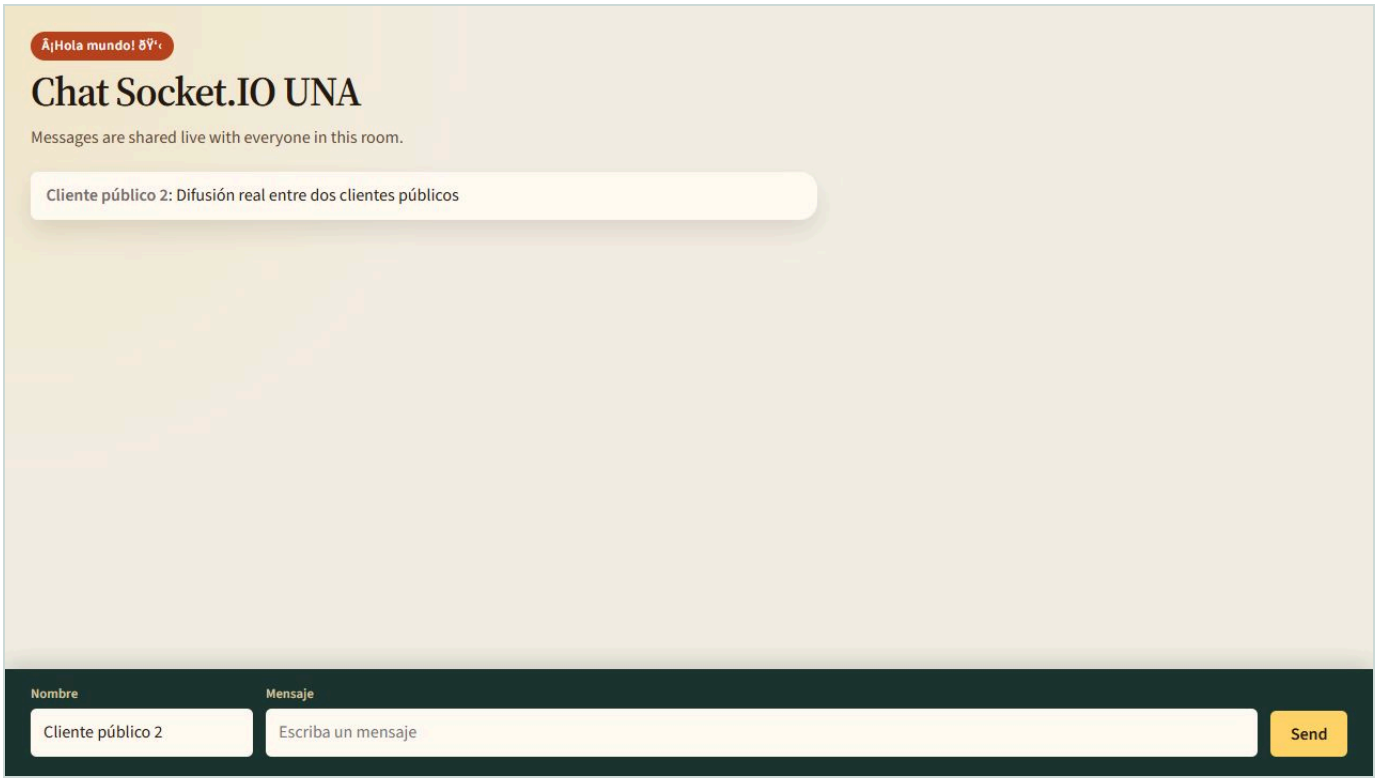


Captura original de la aplicación desde el alias de producción. La evidencia muestra el chat activo y mensajes difundidos; no se ejecutó carga contra el servicio público.

04 · SOCKET.IO

Conversación entre dos clientes

Se usaron dos páginas independientes del alias público. Ambas establecieron el transporte WebSocket y recibieron los mensajes publicados en la sala.



Comprobación funcional

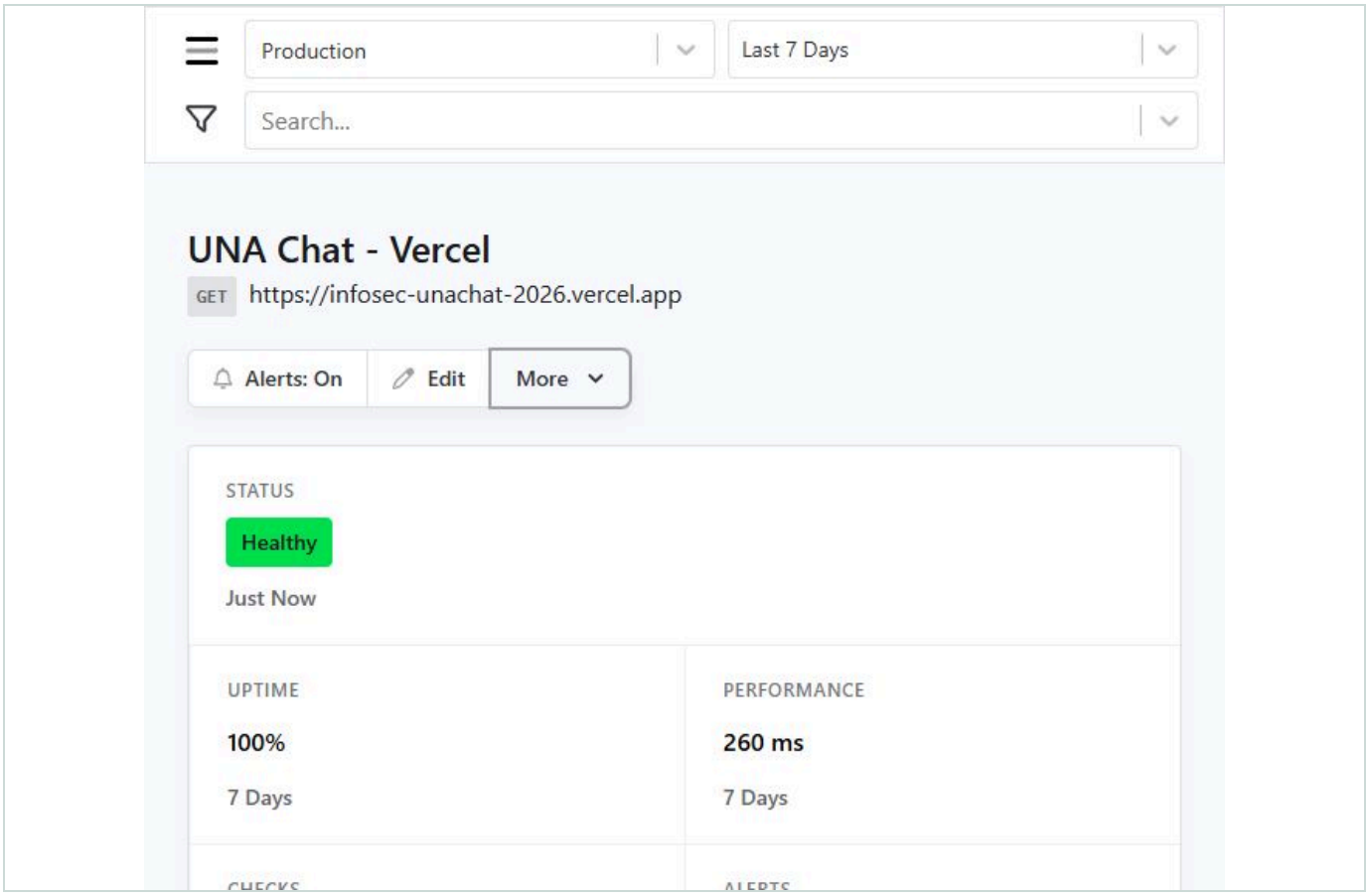
- Cliente 1 envió “Socket.IO en alias público de Vercel”.
- Cliente 2 envió “Difusión real entre dos clientes públicos”.
- La interfaz mostró los mensajes; se verificó el transporte WebSocket en ambos clientes.

Registro: `parte-1-vercel-sentry-cronitor/vercel-chat-test-2026-10-30.txt`

05 · CRONITOR

Disponibilidad y notificaciones

Se creó el monitor **UNA Chat - Vercel** para el alias público. La configuración es GET cada cinco minutos, timeout de 10 segundos, cuatro regiones y lista de notificaciones **Default**. Las alertas están habilitadas.



Resultado observado

Primer chequeo real: Healthy HTTP 200, 260 ms desde Frankfurt. Uptime inicial mostrado: 100% para el período registrado.

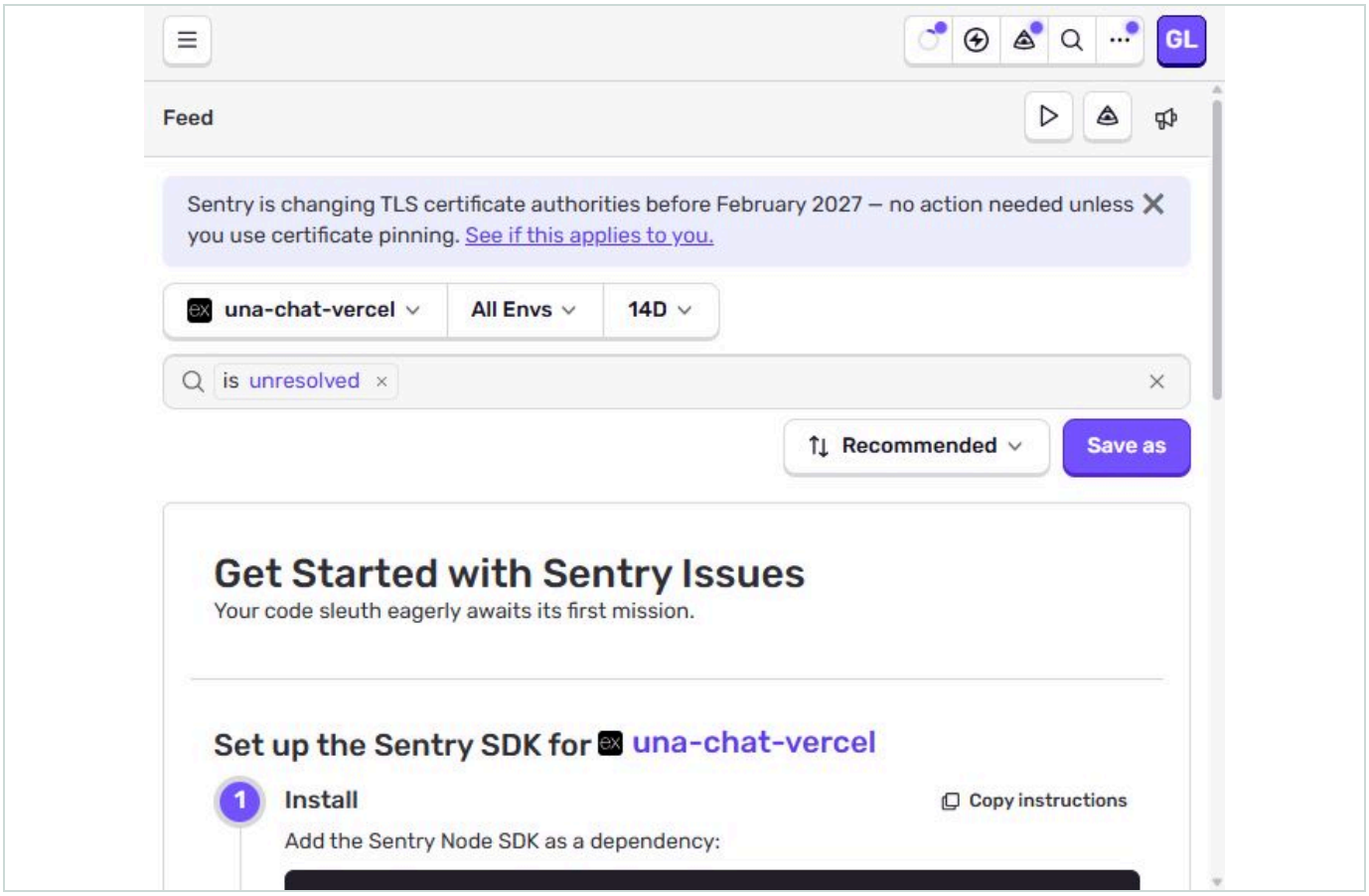
Monitor: cronitor.io/app/monitors/UTzIWf. El monitor anterior de Render se conservó separado.

Las notificaciones están configuradas en Default y el control de alertas indica On. No se generó una caída artificial para probar el envío de una notificación.

06 · SENTRY

Proyecto de captura de errores

En la organización `gerald-aw0` se creó el proyecto Express **una-chat-vercel**. El código de la aplicación inicializa Sentry en Vercel con el SDK Node; la captura de PII está desactivada.



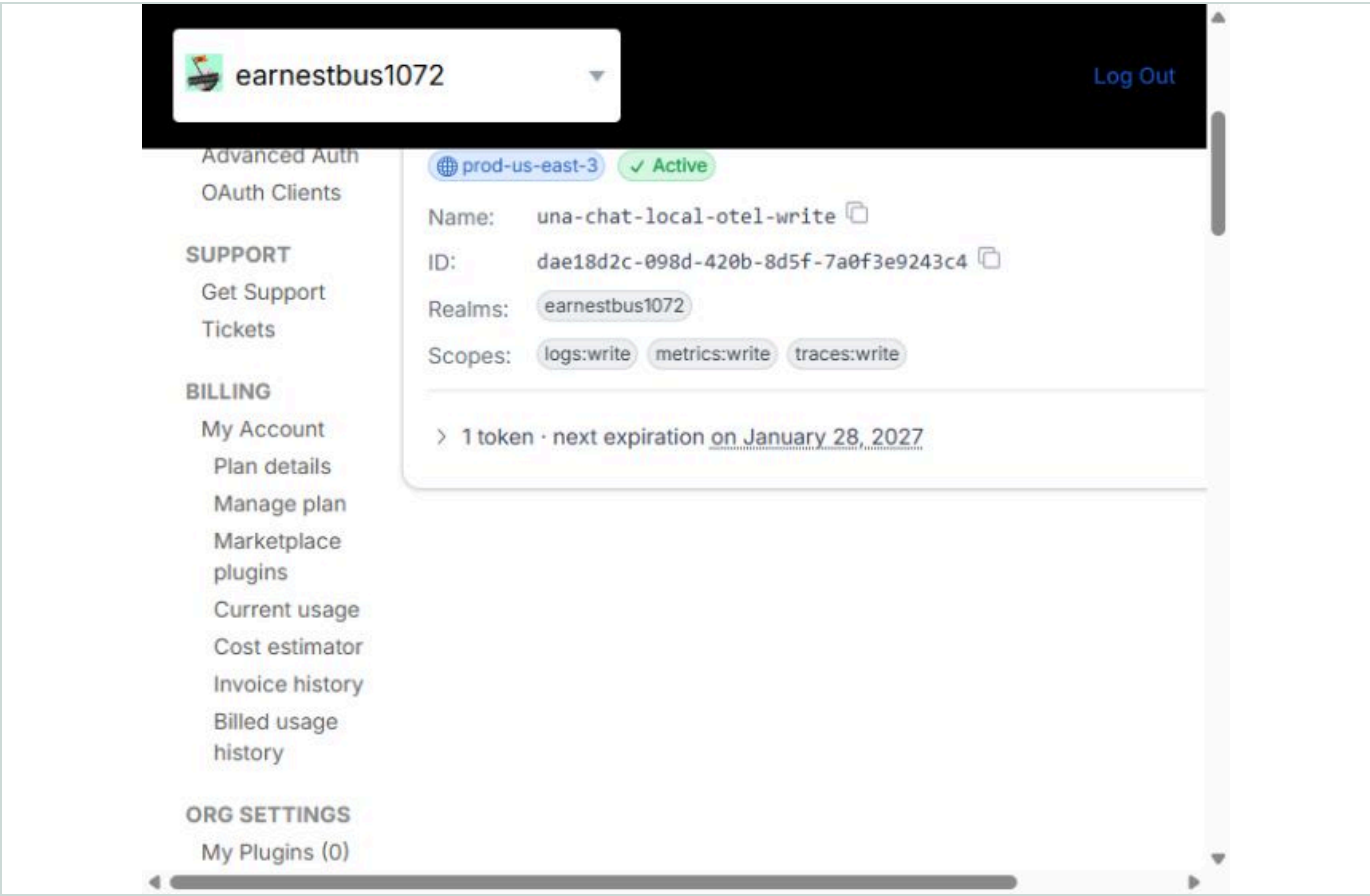
> **Estado pendiente:** la vista real de Issues aún muestra el onboarding sin eventos. Falta configurar `SENTRY_DSN` como variable secreta en Vercel, generar el error controlado protegido y verificar el evento. Por tanto, no se afirma recepción de errores.

La evidencia no incluye DSN, claves ni datos de usuario. El mecanismo de prueba está protegido por secreto y debe apagarse tras validar el evento.

07 · GRAFANA CLOUD

Permisos OTLP y señales

Se configuró la política `una-chat-local-otel-write` en la organización Grafana. Sus únicos permisos son `logs:write`, `metrics:write` y `traces:write`.



Separación por entorno

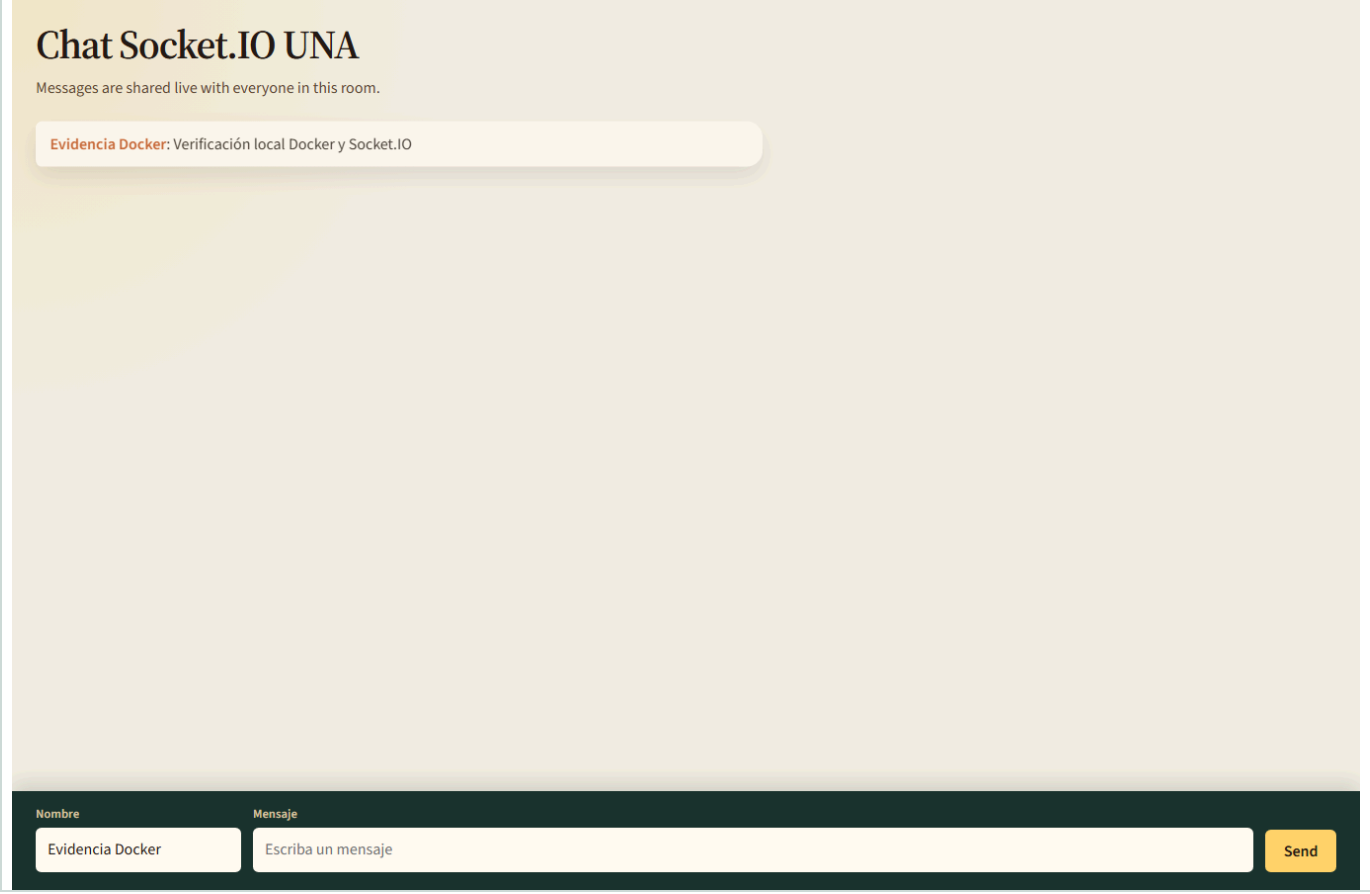
- El endpoint OTLP se usa desde el proceso local/Docker.
- La imagen construye el encabezado de autorización desde `GRAFANA_CLOUD_OTLP_TOKEN` y descarta la variable en el arranque.
- La configuración impide exportar telemetría cuando se ejecuta bajo Vercel.

> **Pendiente:** validar la credencial dentro del `.env` ignorado por Git y consultar realmente logs, métricas y trazas en Grafana Cloud. La captura demuestra permisos, no ingestión.

08 · DOCKER

UNA Chat en contenedor local

Docker Engine 29.9.2 respondió a `docker info` y `docker version`. El servicio de Compose quedó Healthy en `127.0.0.1:3000`, sin detener ni modificar contenedores de otros proyectos.



Pruebas locales

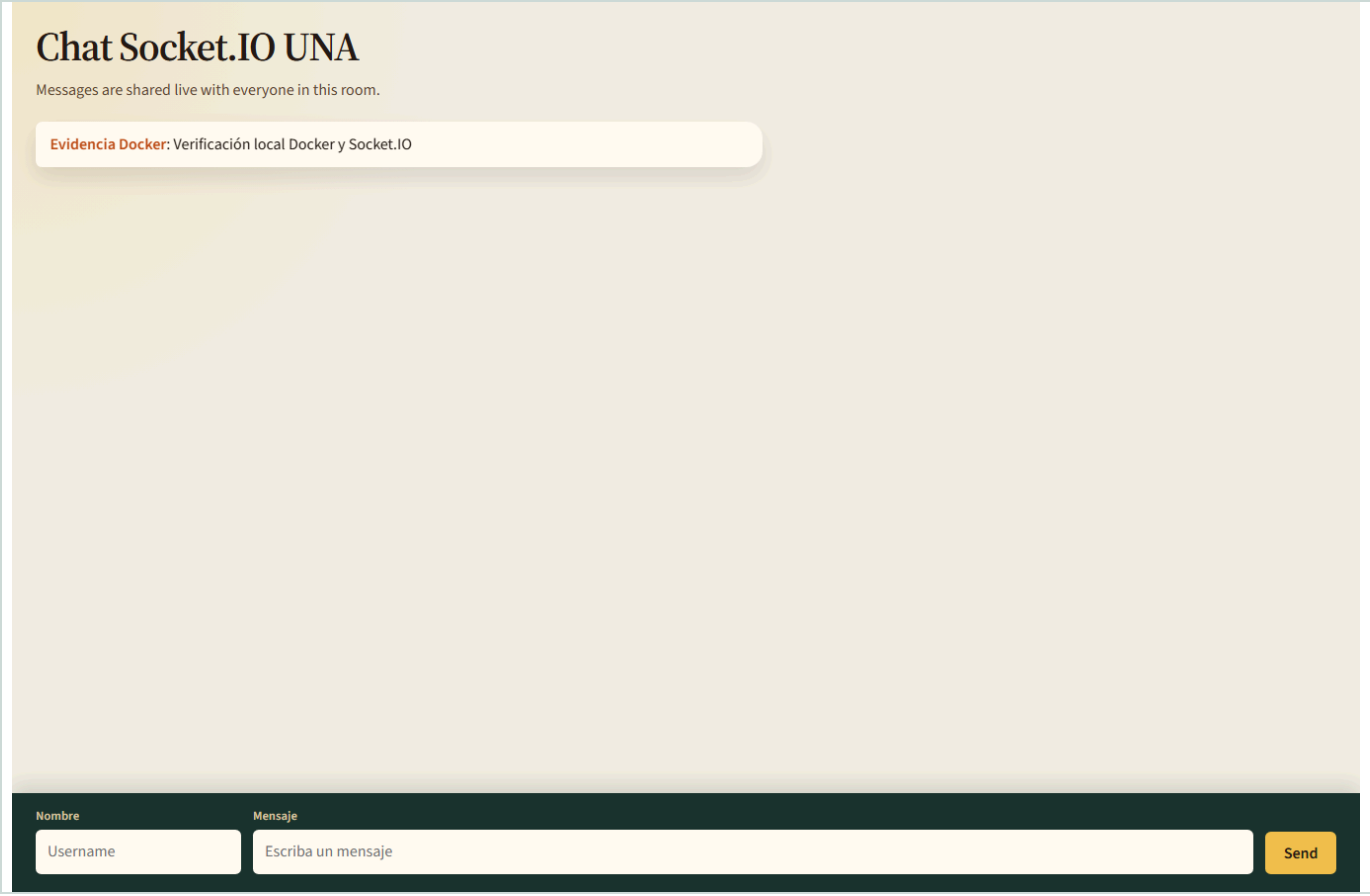
- GET `http://127.0.0.1:3000/` → HTTP 200.
- Node.js 24.x dentro del contenedor.
- Socket.IO difundió mensajes entre dos clientes locales.
- Logs Pino generaron `trace_id` y `span_id` para los mensajes; se omitió el texto del chat.

Configuración reproducible: `Dockerfile`, `docker-compose.yml` y `docker-entrypoint.sh`.

09 · PRUEBA FUNCIONAL

Dos clientes dentro de Docker

La prueba funcional conectó dos páginas al chat de localhost. Un cliente publicó el mensaje mediante el evento Socket.IO real y el otro lo recibió.



Observación de carga externa del equipo

El equipo también mantenía activo el contenedor ajeno `respondent-community-respondent-1`. Se dejó sin cambios. El uso de CPU varió durante la sesión, por lo que los percentiles deben interpretarse como medición del entorno de laboratorio, no como benchmark aislado de hardware.

Carga local: 5 usuarios por segundo

Artillery 2 usó su motor nativo `socketio` durante 60 s con `arrivalRate: 5`, un mensaje por usuario virtual. Se contaron los mensajes procesados por logs de servidor.

Métrica	Socket.IO	HTTP local
Duración / tasa	60 s / 5 VU/s	60 s / 5 VU/s
Solicitudes previstas	300 mensajes	300 POST
Procesadas / respuestas	300 procesados	300 HTTP 200
Errores	0	0
p50	3.7 ms	4.0 ms
p95	17.3 ms	8.9 ms
p99	29.1 ms	22.9 ms

Socket.IO: 300 mensajes emitidos, 300 procesados con `outcome=success`, 0 VUs fallidos. HTTP: adaptador local-only `/__lab/message`, 300 respuestas 200. No se inventaron códigos HTTP para eventos Socket.IO.

Lugar de ejecución: solo `http://127.0.0.1:3000`. Reportes en `artillery-socketio-retest-2026-10-30.txt` y `artillery-http-retest-path-2026-10-30.txt`.

11 · VALIDACIÓN Y CIERRE

Pruebas, reproducción y conclusión

Suite de aplicación

Suite (Node 24.22.1)	Resultado
Mocha	12 passing
Jest integración	5 passing
Playwright E2E	6 passing
Total	**23/23 passing**

La auditoría de producción reportó 0 vulnerabilidades. La auditoría completa posterior a Mocha 12 quedó con 3 bajas y 24 moderadas, sin altas. La instalación local completa de npm quedó lenta en Windows; en Docker sí se completó `npm ci --omit=dev`.

Comandos principales

```
docker compose up -d --build
npm run test:load:socketio
npm run test:load:http
curl http://127.0.0.1:3000/
```

Conclusión

El chat público y la ejecución local contenerizada funcionan; Cronitor registra la disponibilidad y Artillery procesó los 300 mensajes previstos. Para cerrar todas las partes académicas aún deben completarse el evento de Sentry desde Vercel y la recepción/consulta de las tres señales en Grafana Cloud.

Anexo · índice de capturas originales

Vercel: 01-vercel-chat-funcionando; 02-vercel-dos-clientes. Cronitor: 03-cronitor-render-healthy; 04-cronitor-vercel-monitor-created; 05-cronitor-vercel-healthy. Sentry: 06-sentry-project-no-events-yet. Grafana: 01-grafana-politica-otlp-scopes. Docker: 01-chat-docker; 02-conversacion-docker-dos-clientes; 03-chat-docker-verificado; 04-recepcion-dos-clientes-docker. Todas llevan fecha 2026-10-30 y están en `evidencias-laboratorio/`.

Informe extendido: `ENTREGA-LABORATORIO.md`. Índice y metadatos: `evidencias-laboratorio/README.md`. Token OTLP y DSN excluidos.